

[78] HQANN: Efficient and Robust Similarity Search for Hybrid Queries with Structured and Unstructured Constraints

요약

Wu et al.의 CIKM 2022 논문으로, 구조화된 제약(structured constraints, 범위 필터)과 비구조화된 제약(unstructured constraints, 벡터 검색)을 모두 가진 하이브리드 쿼리 처리를 다룬다. 기존 시스템들이 이러한 이질적 제약을 분리하여 처리했다면, HQANN은 통합적 접근을 제안한다.

핵심 기여: 1. **제약 결합 메커니즘**: 구조화된 필터와 비구조화된 벡터 조건을 함께 고려하는 검색 알고리즘 2. **적응적 전략**: 데이터와 쿼리 특성에 따라 처리 순서 동적 결정 3. **강건성(Robustness)**: 극단적 선택도(매우 높거나 매우 낮음)에서도 안정적 성능 유지

알고리즘은 두 제약의 선택도를 추정하고, 이를 기반으로 최적의 처리 전략을 선택한다. 또한 검색 과정에서 동적으로 탐색 경로를 조정하여 원하는 결과를 효율적으로 찾는다.

본 논문과의 관계: HQANN은 Exqutor가 직면하는 정확히 같은 문제를 다룬다. 범위 필터(structured)와 벡터 검색(unstructured)의 결합에서 최적 성능을 달성하는 방법론을 제시한다. 특히 선택도 추정을 통한 적응적 전략 선택이 Exqutor의 핵심 메커니즘과 부합한다.

상세분석

78.1 하이브리드 쿼리 문제의 정의

쿼리 예시:

```
SELECT TOP 10 *
FROM products
WHERE (price >= 100 AND price <= 500) -- 구조화된 제약
      AND SIMILARITY(embedding, query_vector) >= 0.8 -- 비구조화된 제약
ORDER BY SIMILARITY(embedding, query_vector) DESC
```

두 제약의 특성:

측면	구조화된 제약	비구조화된 제약
형태	범위, 등호, 부등호	벡터 유사도 비교
효율성	$O(\log n) \sim O(n)$	$O(n \log n)$ (ANN)
정확도	100%	~95% (recall 기준)
선택도	예측 가능	데이터 분포에 의존
인덱스	B-Tree, Hash	HNSW, IVF

78.2 하이브리드 쿼리 처리 전략

전략 1: 순차적 처리 (Sequential)

- (A) 필터 먼저: Filter-then-Search
범위 필터 → ANN 검색
비용 = 필터 스캔 + $(1 - \sigma_{\text{filter}})N \times \text{ANN_검색}$
- (B) 검색 먼저: Search-then-Filter
ANN 검색 → 범위 필터
비용 = ANN_검색 + 필터_검사 $\times K$

전략 2: 병렬 처리 (Parallel)

필터 탐색 ↙
└─ 병합 및 정렬
검색 탐색 ↘

비용 = $\max(\text{필터_시간}, \text{ANN_시간}) + \text{병합_시간}$

전략 3: 통합 처리 (Integrated)

하나의 통합 인덱스에서 두 제약을 동시에 적용
- HQANN의 접근

78.3 HQANN의 핵심 알고리즘

3.1 이중 경로 탐색(Dual-Path Search)

```

def hqann_search(query_vector, range_filter, top_k):
    # 1단계: 양쪽 제약 모두 만족하는 후보 찾기
    candidates = []
    visited = set()

    # 초기 진입점: 필터 조건 근처에서 시작
    entry_points = find_entry_points(range_filter)

    # 2단계: 그래프 탐색 (HNSW 기반)
    priority_queue = PriorityQueue()
    for ep in entry_points:
        priority_queue.add(ep, distance=0)

    while len(candidates) < top_k and not priority_queue.empty():
        node = priority_queue.pop()

        if node in visited:
            continue
        visited.add(node)

        # 양쪽 제약 모두 확인
        if satisfies_range_filter(node, range_filter) and \
            is_good_for_vector_similarity(node, query_vector):
            candidates.add(node)

        # 이웃 탐색
        for neighbor in get_neighbors(node):
            if neighbor not in visited:
                dist = compute_distance(query_vector, neighbor)
                priority_queue.add(neighbor, distance=dist)

    return candidates[:top_k]

```

3.2 선택도 기반 진입점 선택

range_filter에서 σ_r (범위 필터 선택도) 계산

$\sigma_r < 10\%$:

- └ 범위 필터 만족하는 벡터에서 시작
(필터링된 인덱스 사용)

$\sigma_r > 50\%$:

- └ 전체 벡터에서 유사도 기반 시작
(벡터 유사도 인덱스 사용)

$10\% < \sigma_r < 50\%$:

- └ 두 조건의 균형 잡기
(하이브리드 진입점 선택)

78.4 강건한 전략 선택

선택도 쌍을 이용한 전략 결정:

σ_r : 범위 필터 선택도

σ_v : 벡터 유사도 선택도 (top-k 반환 비율)

경우 1: σ_r 매우 낮음 ($< 5\%$), σ_v 높음 ($> 50\%$)

추천: Filter-then-Search

이유: 필터로 대부분 제거 가능, 남은 것에서만 검색

경우 2: σ_r 높음 ($> 80\%$), σ_v 낮음 ($< 20\%$)

추천: Search-then-Filter

이유: 벡터 검색으로 상위 k 찾고 필터 확인

경우 3: σ_r 중간 (20-80%), σ_v 중간 (30-70%)

추천: Integrated HQANN 전략

이유: 순차 처리의 단점이 클 수 있음

경우 4: 하나는 매우 높고 하나는 매우 낮음

추천: 조건부 필터-검색 (Conditional Filter-Search)

이유: 낮은 선택도 조건 먼저, 높은 선택도 조건은 확인만

78.5 동적 탐색 조정

런타임 피드백 루프:

초기 전략 선택 (선택도 기반)

↓

일정 수의 쿼리 결과 탐색

↓

실제 선택도 측정

↓

예상과 실제 선택도 비교

↓

- └ 차이 크면: 탐색 방향 조정
- └ 예상 선택도보다 높음: 더 탐색
- └ 예상 선택도보다 낮음: 탐색 확장

예시:

예상 선택도: $\sigma_r = 15\%$

초기 전략: Filter-then-Search

탐색 중:

- 처음 100개 노드 방문: 15개가 범위 필터 만족 ✓
- 다음 100개 노드 방문: 5개만 범위 필터 만족 (예상 15개)

발견: 예상보다 낮은 선택도 → 더 많이 탐색 필요

조정: 탐색 반경 확대

78.6 혼합 인덱스 구조

다중 인덱싱 방식:

방식 1: 독립 인덱스 (Independent)

- └ 범위 필터 인덱스: B-Tree (price)
- └ 벡터 인덱스: HNSW

문제: 두 인덱스 조율 어려움

방식 2: 계층 인덱스 (Hierarchical)

- └ 범위 필터로 1차 분할
- └ 각 분할 내에서 벡터 인덱스

문제: 범위 필터 변경 시 구조 재구성 필요

방식 3: 통합 인덱스 (Unified - HQANN)

- └ 메인 HNSW 그래프
- └ 각 노드에 범위 필터 메타데이터 첨부
- └ 범위 만족 노드만 우선적으로 방문

이점: 유연한 필터 조건, 캐시 효율

78.7 실험 결과

벤치마크 설정: - 데이터셋: Yahoo Products (10M), StackOverflow (1M) - 쿼리: 다양한 (σ_r , σ_v) 조합 - 비교: Filter-then-Search, Search-then-Filter, HQANN

결과 (쿼리 응답 시간, ms):

(σ_r , σ_v)	FTS	STF	HQANN	최적	HQANN 효율
(5%, 70%)	30	500	35	30	117%
(15%, 50%)	100	300	110	100	109%
(50%, 50%)	400	400	410	400	102%
(80%, 20%)	700	150	160	150	107%
(90%, 5%)	1500	80	85	80	106%

극단적 경우에서의 강건성:

(σ_r , σ_v) = (1%, 99%): 거의 모든 벡터가 조건 만족
FTS: 매우 많이 탐색 필요 (800ms)
STF: 빠른 검색 후 대부분 필터 통과 (100ms)
HQANN: 벡터 유사도 기반 탐색으로 빠르게 결과 (110ms)

(σ_r , σ_v) = (99%, 1%): 거의 모든 벡터가 필터 통과
FTS: 빠른 필터 후 거의 대부분 검색 (900ms)
STF: 빠른 검색 후 거의 모두 필터 통과 (800ms)
HQANN: 범위 필터 기반 진입점으로 효율적 탐색 (750ms)

78.8 선택도 추정의 중요성

정확한 선택도 추정이 필수:

전략 선택은 선택도 추정에 의존

시나리오: 실제 (σ_r , σ_v) = (20%, 50%)

추정: (20%, 50%) → HQANN 선택 → 110ms ✓

추정: (5%, 70%) → FTS 선택 → 400ms ✗

추정: (80%, 20%) → STF 선택 → 200ms (약 50% 저하)

78.9 본 논문과의 관계

Exqutor의 핵심 문제와 HQANN의 솔루션:

Exqutor가 해결해야 하는 문제:

범위 필터: WHERE price $\in$ [L, U]

벡터 검색: WHERE similarity(embedding) > threshold

두 조건의 효율적 결합?

HQANN의 답변:

1. 선택도 추정 (Exqutor 논문 [70])

σ_r = selectivity_model.estimate(price_range)

2. 적응적 전략 선택

if $\sigma_r < 10\%$:

 사용 Filter-then-Search

elif $\sigma_r > 80\%$:

 사용 Search-then-Filter

else:

 사용 HQANN 통합 처리

3. 동적 조정

 실제 탐색 중 선택도가 예상과 다르면 조정

Exqutor와 HQANN의 통합:

Exqutor 아키텍처:

 쿼리 분석

 ↓

 범위 필터 선택도 추정 (ML 모델)

 ↓

 벡터 유사도 특성 파악 (데이터셋 통계)

 ↓

 전략 선택 (HQANN 원리)

 ↓

 범위 필터 + 벡터 검색 실행

 ↓

 결과 반환

추가 제기 문제

1. 선택도 추정 오류의 영향: 선택도 추정이 $\pm 50\%$ 오류일 때, 부정확한 전략을 선택할 확률은?

2. **다중 필터 조건:** AND, OR로 결합된 여러 범위 필터가 있을 때, 선택도를 정확히 계산할 수 있는가?
3. **실시간 선택도 추정:** 쿼리별로 선택도를 추정하는 오버헤드는 얼마나 되는가?
4. **캐시 워밍:** 자주 나오는 (range_filter, query_vector) 쌍에 대해 캐싱할 수 있는가?
5. **동적 전략 전환:** 탐색 중간에 전략을 바꿀 때의 오버헤드와 이득의 균형은?
6. **상관성 처리:** 범위 필터 속성(가격)과 임베딩 차원 사이의 상관성이 있을 때 어떻게 활용할 것인가?